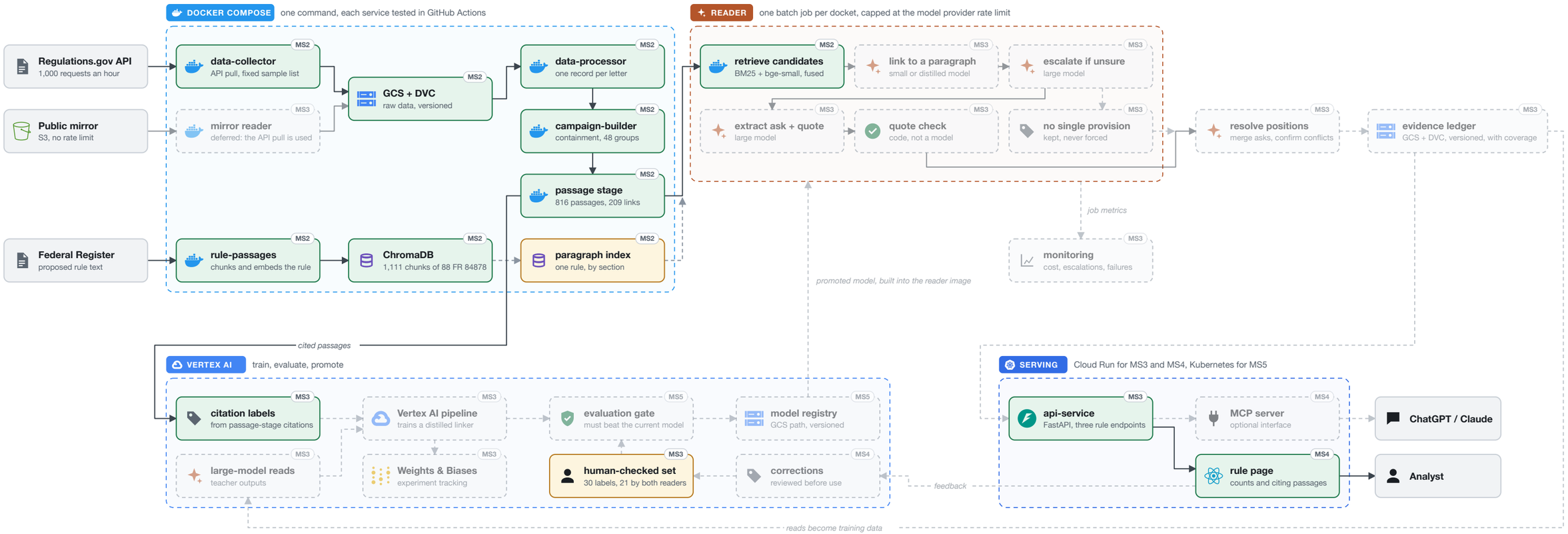
Undersigned: the design, and where it stands on 16 October 2026

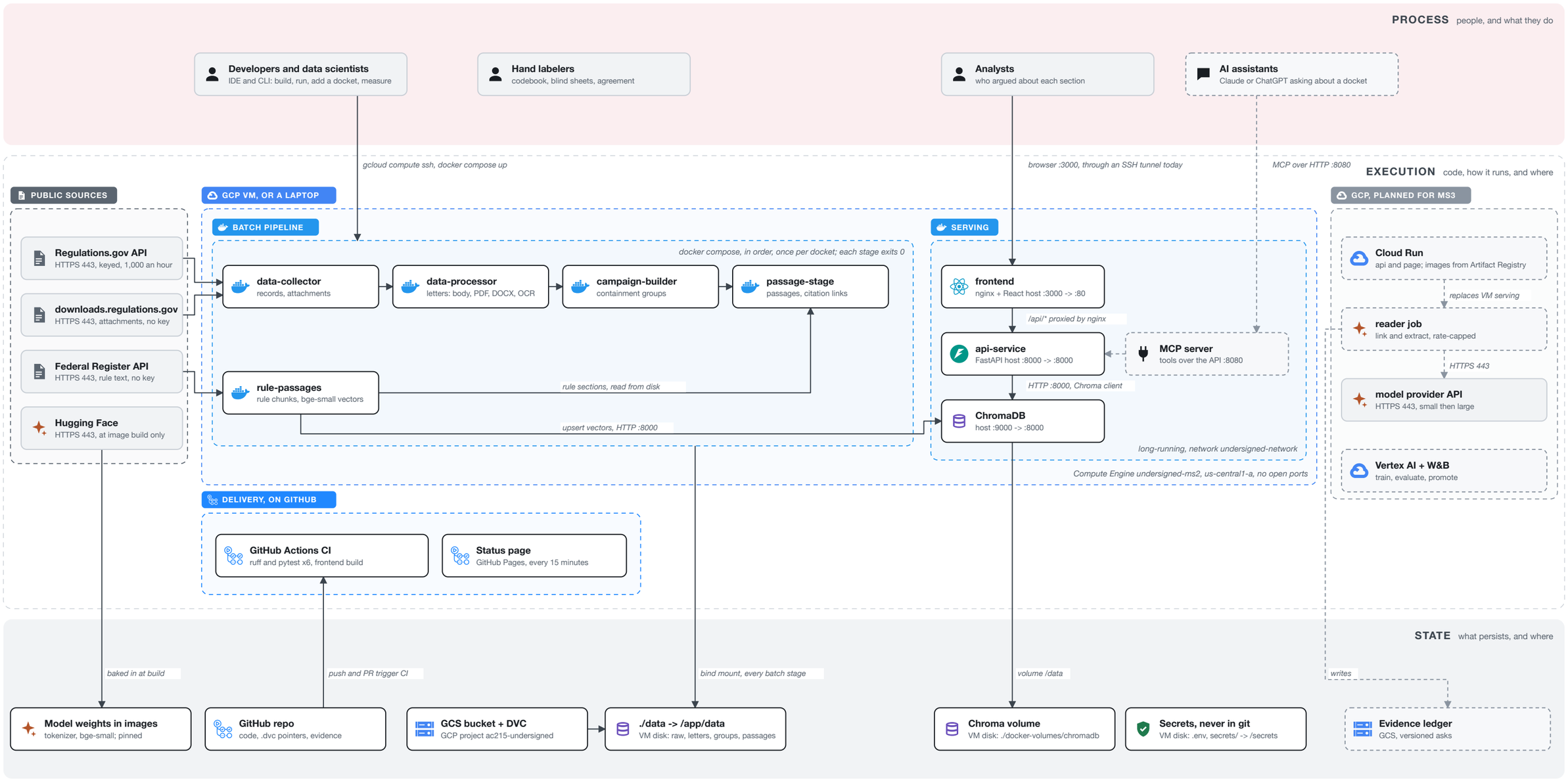
Read every substantive letter on a rule once, as a batch job, record each argument against the paragraph it concerns, and serve the record. Green runs in main today from one command on a fresh clone. Amber is partly there, and the box says what exists. Faded is not started.



As of 16 Oct 2026. Green: in main and runs in the one-command chain. Amber: partly there, the box says what exists. Faded: not started in the repository. Grey fill: outside the system.

Undersigned solution architecture

Who uses it, the code that runs, how and where it is hosted, and what persists. Ports, mounts and endpoints are read from docker-compose.yml, the Dockerfiles, frontend/nginx.conf and the code on main; hosting from docs/setup.md and the VM run. 16 October 2026.



Solid border: runs in main today. Dashed: planned. Arrow labels: protocol and port. Host ports: 3000 page, 8000 API, 9000 ChromaDB; batch containers expose none. The arrow between the bucket and ./data is dvc pull and push, HTTPS 443.

Layers, per container

CONTAINER	RUNS	BAKED INTO THE IMAGE AT BUILD	GIVEN AT START: SETTINGS, SECRETS, MOUNTS	FETCHED WHILE RUNNING	OUTPUT	REACHED AT
data-collector	batch, exits	Python 3.12 and its libraries	`.env` : data.gov key, docket, sample list. Mounts `./data` , the sample list (read-only), `secrets/`	records and attachments from Regulations.gov, HTTPS 443	`data/raw/<docket>/`	no port
data-processor	batch, exits	locked libraries, Poppler, Tesseract OCR, bge tokenizer (checksum-pinned)	`.env` : signer hash key, docket, sample list. Mounts `./data` , sample list, `secrets/`	nothing	`data/derived/letters/`	no port
campaign-builder	batch, exits	locked libraries	`.env` : docket, grouping method. Mounts `./data` , `secrets/`	nothing	`data/derived/campaigns/`	no port
passage-stage	batch, exits	locked libraries, rule-passages' chunking code	`.env` : docket. Mounts `./data`	nothing; reads the rule text rule-passages saved and the groups campaign-builder wrote	`data/derived/passages/`	no port
rule-passages	batch, exits	locked libraries, bge-small embedding model	`.env` : docket, ChromaDB address. Mounts `./data` , `secrets/`	the proposed rule from the Federal Register, HTTPS 443	rule text on disk; vectors into ChromaDB	no port
api-service	long-running	locked libraries, bge-small, rule-passages' search code	`.env` , ChromaDB address. Mounts `secrets/` , `./data/derived/passages` (read-only)	the rule index from ChromaDB, once at startup	JSON: `/health` , `/retrieve` , `/rules/{docket}/sections` , `/rules/{docket}/passages`	**:8000**
frontend	long-running	the React page, built with Node, served by nginx; the proxy config	nothing	API calls on each page load, through `/api/`	the rule page	**:3000** → 80
chromadb	long-running	pulled image `chromadb/chroma:1.5.9`	volume `./docker-volumes/chromadb` → `/data`	nothing	vector search over HTTP	**:9000** → 8000

Access. On the VM no port is open to the internet: a browser reaches the page on 3000 and the API's docs on 8000 through `gcloud compute ssh -L` . On a laptop the same ports are on localhost. 9000 is for developers inspecting ChromaDB. **Secrets.** `.env` and `secrets/` are git-ignored and injected when a container starts, never baked into an image. No container uses Google Cloud today. DVC runs outside the containers: on the VM it authenticates as the VM's service account, on a laptop with the shared key file. **Code.** Each Python container also mounts its own `src/` over the baked code, so an edit applies without a rebuild. Shaded cells are fetched over the network while running. Everything else is fixed at build or start.